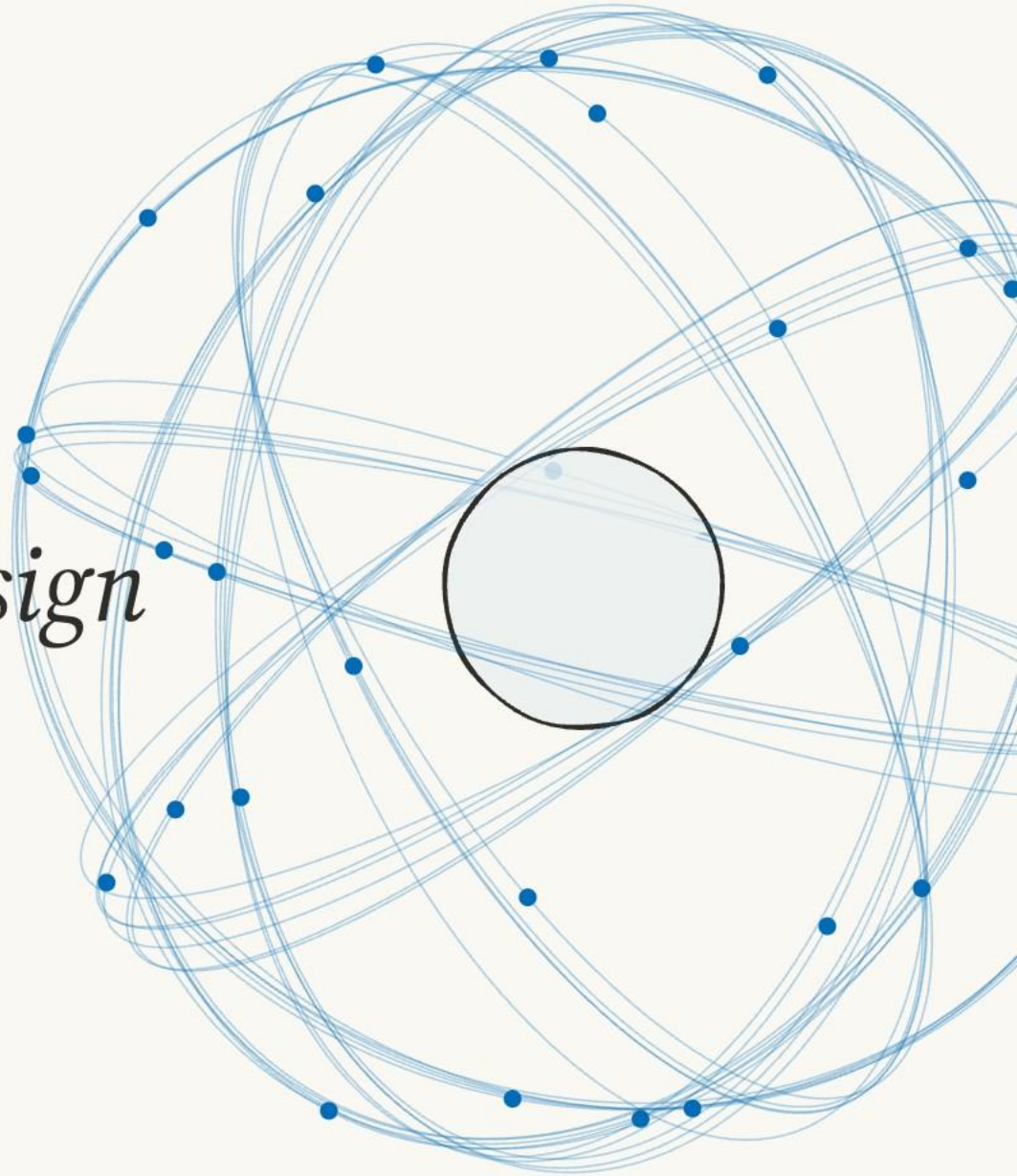
Practical Astrodynamics: *Open-Source Tools for Constellation Analysis and Design*

Duncan Eddy, PhD

2026 SPNT SYMPOSIUM · NOVEMBER 12, 2026



Stanford University

GPS satellites on November 12, 2026, from the IGS ultra-rapid orbit IGS0OPSULT_20263150000_02D_15M_ORB

Which GPS satellites are above this building right now?

Which GPS satellites are above this building right now?

Basic

- Keplerian two-body orbits
- Earth spun about its axis
- Spherical Earth, geocentric latitude

Median position error

23.5 km

Better

- TLEs with SGP4
- TEME to Earth-fixed, with Earth orientation parameters
- Geodetic station position

Median position error

1.7 km

Full

- IGS SP3 orbit products, choosing the best one
- Earth-fixed positions, time-tagged in GPS time
- Geodetic station on the reference ellipsoid

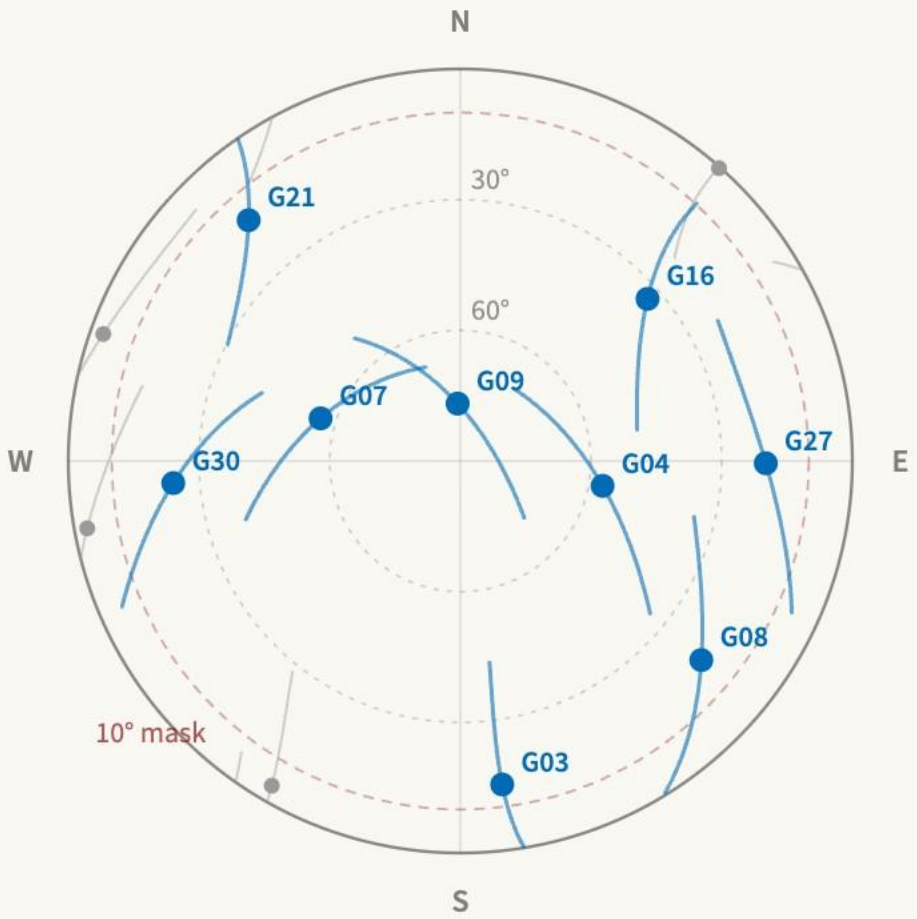
Median position error

3.2 cm

Ultra-rapid prediction against the IGS rapid orbit, 7 days earlier

IGS ultra-rapid orbit IGS0OPSULT_20263150600_02D_15M_ORB · GP elements from Space-Track · ultra-rapid against rapid orbit on November 5 · scripts/three_levels.py

Which GPS satellites are above this building right now?




```python
import brahe as bh

GEODETIC = bh.EllipsoidalConversionType.GEODETIC
epoch = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
lla = [-122.1697, 37.4275, 30.0]  # Stanford lon, lat, alt

igs = bh.IGSProductsClient("BKG")
ultra_rapid = igs.list_products_between(
    epoch, epoch, center="IGS", solution="ULT", content="ORB"
)
sp3 = igs.download_sp3(ultra_rapid[-1])  # latest issue
site = bh.position_geodetic_to_ecef(lla, bh.AngleFormat.DEGREES)

visible = []
for sat in sp3.satellites():
    r = sat.state_ecef(epoch)[:3]
    enz = bh.relative_position_ecef_to_enz(site, r, GEODETIC)
    _, el, _ = bh.position_enz_to_azel(enz, bh.AngleFormat.DEGREES)
    if sat.get_name().startswith("G") and el >= 10.0:
        visible.append(sat.get_name())
```

9 GPS satellites above 10° elevation at Stanford,
11:00 a.m. Pacific time on November 12, 2026

IGS ultra-rapid orbit IGS0OPSULT_20263150000_02D_15M_ORB at 2026-11-12 19:00 UTC · snippets/skyplot.py · full analysis in scripts/skyplot.py

Improving Navigation by Removing Errors

Measurement error

Clocks, ionosphere, troposphere, multipath, receiver noise

System design changes can also reduce measurement error

Much of the dynamics can be modeled, but it is hard to get right and hard to make fast.

Dynamics model error

Uncertainty in location from unmodeled forces, and from uncertainty and errors in capturing the true physics

Earth's gravity field beyond a point mass

Atmospheric drag

Solar radiation pressure

Sun and Moon gravity

Solid Earth tides and relativity

The time systems and frames all of these are expressed in

Astrodynamics tools tend to trade these properties off against each other

	Capable	Fast	Easy to use	Open
Commercial tools STK, FreeFlyer	✓	~	✗	✗
Research scripts and small packages university and personal projects	~	✗	✗	✓
Established open-source toolkits Orekit, GMAT	✓	✗	✗	✓

✓ yes · ~ somewhat · ✗ no

General tendencies, not a ranking of any one tool

"It has always been too much astrodynamics for software engineers, and too much software engineering for astrodynamicists."

Juan Luis Cano Rodríguez, "Juan Luis steps down as maintainer of the project", poliastro blog, December 21, 2022

https://www.poliastro.space/blog/2022/12/21/juan-luis-steps-down/

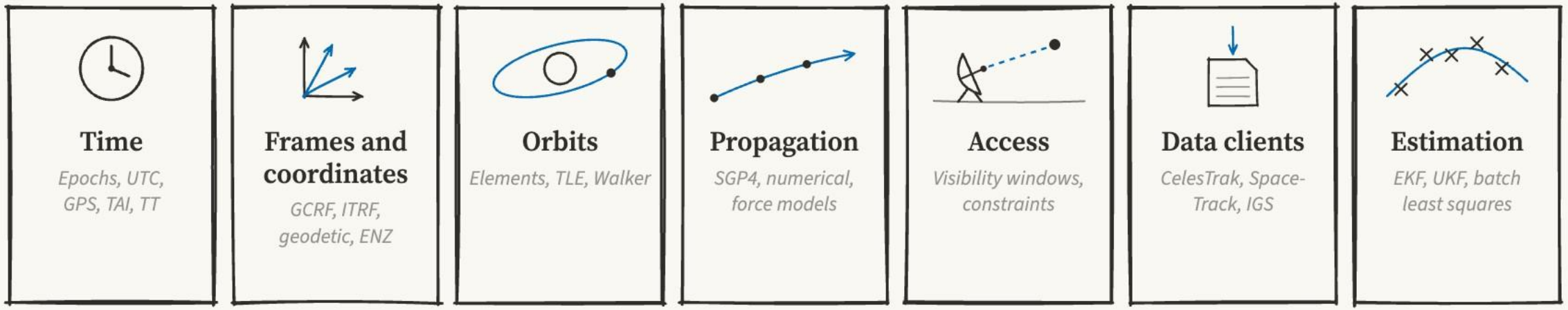


brahe is an open-source astrodynamics project that aims to solve this problem: to empower engineers and researchers to use the highest-fidelity models, with the speed of simpler ones, without adding complexity.

Fast	Cross-platform	Free
Rust core, Python bindings	Linux, macOS and Windows	MIT licensed

```
pip install brahe
```

The modules in brahe



I have rewritten this project five times since 2014



2014 Start GNC research at Stanford and find no good open-source software to use. Start building my own in **MATLAB**.

2015 to 2016 Find MATLAB is too slow for my use cases. Rewrite in **C++ with MEX bindings**.

2017 Join Capella Space. Rewrite in **pure C++**.

2018 C++ becomes too much of a pain to write quickly and correctly. Rewrite in **pure Python**.

2019 Return to Stanford. Rewrite in **Julia**.

2021 Join Amazon Kuiper. Start rewriting in **Rust with Python bindings**, which is what it has been since.

Design choices in brahe

SI units everywhere

Meters, seconds and kilograms in and out of every function. Angles state degrees or radians.

Composable pieces

Propagators, force models, access constraints and event detectors are separate objects you combine.

Make users explicitly invoke magic

You state each model choice precisely. Once invoked, brahe handles the complexity of using it.

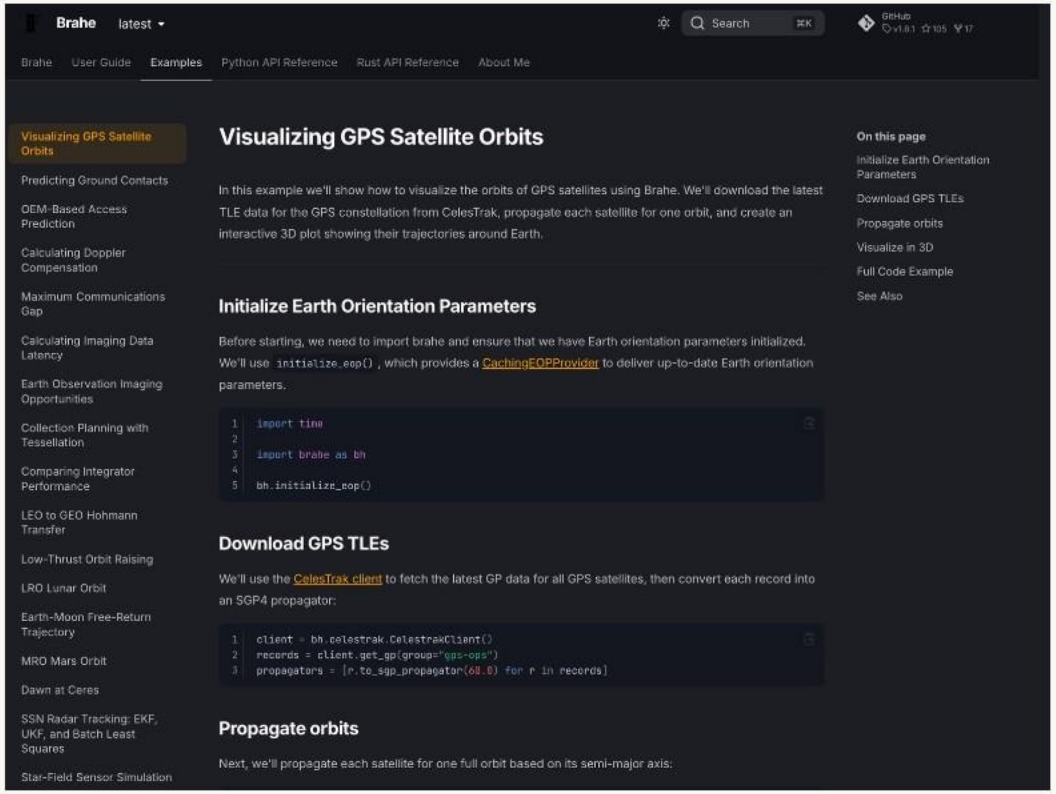
Performance optimization abstracted

Compiled Rust, threading and whole-array calls sit behind ordinary Python functions.

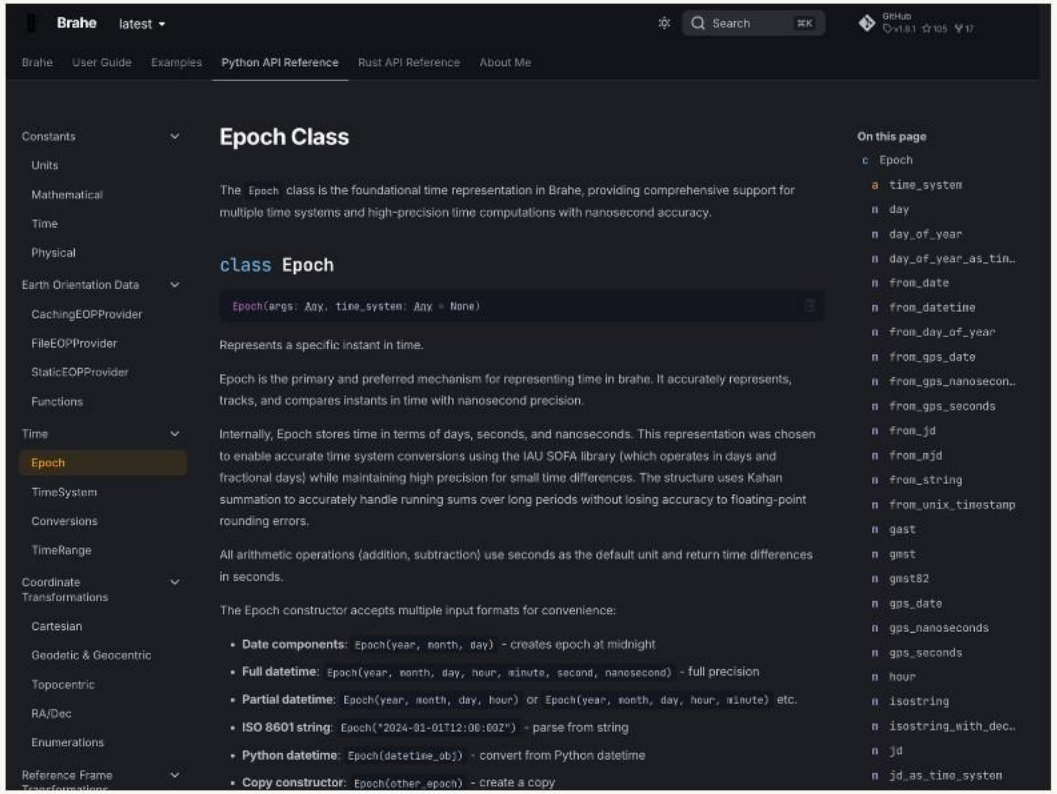
Every documentation example runs

Each example in the documentation is compiled and run on every commit, so the docs match the code.

The documentation has worked examples and a full API reference



Examples: worked problems, start to finish



API reference: every class and function, with examples

docs.brahe.space · brahe 1.8.1 documentation

Time

SUPPORTS

Time systems

UTC, TAI, GPS, TT, UT1
TDB, TCG, TCB
BeiDou (BDT) and Galileo (GST) time

Epoch

Nanosecond resolution, compensated sums
Calendar, JD, MJD, GPS week and seconds, ISO 8601
`TimeRange` grids of epochs

Earth orientation and leap seconds

Leap seconds from SOFA, compiled in
IERS EOP 20 C04 and finals2000A files
UT1−UTC, polar motion, dX and dY, LOD

The same instant in four time scales

```python
import brahe as bh
bh.initialize_eop()
# One instant, defined in UTC
epc = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
# The same instant in four other time scales
for ts in (bh.GPS, bh.TAI, bh.TT, bh.UT1):
    print(epc.to_time_system(ts))
```

```
OUTPUT  2026-11-12 19:00:18.000 GPS                    7.4 ms
        2026-11-12 19:00:37.000 TAI
        2026-11-12 19:01:09.184 TT
        2026-11-12 18:59:59.933 UT1
```

snippets/mod_time.py · output and run time (imports excluded) from scripts/snippet_outputs.py · brahe 1.8.1 documentation, brahe.space



Frames and coordinates

SUPPORTS

Earth frames

GCRF, ITRF, EME2000, MOD, TOD, TEME

Other bodies and interplanetary frames

Moon: LCI, LFPA, LFME; Mars: MCI, MCMF
Barycentric EMBI, SSBI; synodic EMR, SER, GSE
IAU body-fixed frames by NAIF ID
Any to any: `state_frame_to_frame`

Earth rotation models

IAU 2006/2000A, CIO based, from SOFA
IAU 2000B option; GAST and GMST 1982 chains

Coordinates

Keplerian, equinoctial, geodetic (WGS84), geocentric
ENZ, SEZ, azimuth and elevation, RA and Dec
RTN, LVLH and relative orbital elements

100,000 states between frames in one call, and by frame name

```python
import brahe as bh
import numpy as np
bh.initialize_eop()
epc = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
# One state at GPS altitude, repeated 100,000 times
oe = np.array([bh.R_EARTH + 20_200e3, 0.0, 55.0, 0.0, 0.0, 0.0])
x0 = bh.state_koe_to_eci(oe, bh.AngleFormat.DEGREES)
x = np.tile(x0, (100_000, 1))
# Inertial to Earth-fixed, every state in one call
x_itrf = bh.state_gcrf_to_itrf(epc, x)
# Any frame to any frame, by name
F = bh.CelestialFrame
x_teme = bh.state_frame_to_frame(F.GCRF, F.TEME, epc, x)
print(x_itrf.shape, x_teme.shape)
```

```
OUTPUT  (100000, 6) (100000, 6)                    295 ms
```

snippets/mod_frames.py · output and run time (imports excluded) from scripts/snippet_outputs.py · brahe 1.8.1 documentation, brahe.space

Orbits

SUPPORTS

Elements

Keplerian and equinoctial elements, anomalies
Period, apsides, sun-synchronous inclination
Osculating to mean: Brouwer-Lyddane (first-order J2) or numerical averaging

Element sets

TLE and 3LE parsing; TLE generation
Alpha-5 catalog numbers
GP records (OMM) to SGP4 or numerical propagators

Constellations

Walker Delta and Walker Star generators
A Keplerian, SGP4 or numerical propagator per satellite

A GPS-like Walker constellation from one builder call

```python
import brahe as bh

epc = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
a = bh.R_EARTH + 20_200e3

# 24 satellites, 6 planes, phasing 2, inclination 55 degrees
gen = bh.WalkerConstellationGenerator.builder(
    24, 6, 2, a, 55.0, epc
)

# One propagator per satellite, 60 s step
sats = gen.build().as_keplerian_propagators(60.0)

print(len(sats), "satellites")
```

```
OUTPUT  24 satellites                                    130 µs
```

snippets/mod_orbits.py · output and run time (imports excluded) from scripts/snippet_outputs.py · brahe 1.8.1 documentation, brahe.space



Propagation

SUPPORTS

Propagators

Keplerian, SGP4, numerical orbit, general ODE
Many propagators in parallel in one call
Events: eclipse, nodes, altitude, any function

Force models

Gravity: EGM2008 to 120, GGM05S to 180, JGM3, ICGEM
Drag: NRLMSISE-00, Harris-Priester, exponential
Cannonball SRP, cylindrical or conical eclipse
Sun, Moon, planets (DE440); relativity; tides

Integrators and variational equations

RK4, RKF45, DP54, RKF78, RKN1210
State transition matrix, sensitivity, covariance

One day at GPS altitude with 80×80 gravity, Sun, Moon, relativity and the STM

```python
import brahe as bh
import numpy as np

bh.initialize_eop()
epc = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)

# Initial state at GPS altitude
oe = np.array([bh.R_EARTH + 20_200e3, 0.0, 55.0, 0.0, 0.0, 0.0])
x0 = bh.state_koe_to_eci(oe, bh.AngleFormat.DEGREES)

# Integrator settings and force model, chosen separately
cfg = bh.NumericalPropagationConfig.default().with_stm()
forces = bh.ForceModelConfig.conservative_forces()

# Propagate one day
prop = bh.NumericalOrbitPropagator(epc, x0, cfg, forces)
prop.propagate_to(epc + 86400.0)

r_km = prop.current_state()[:3] / 1e3
print("position (km):", np.round(r_km, 1))
print("STM shape:", prop.stm().shape)
```

```
OUTPUT  position (km): [26570.7   350.4   527.7]          748 ms
        STM shape: (6, 6)
```

snippets/mod_propagation.py · output and run time (imports excluded) from scripts/snippet_outputs.py · brahe 1.8.1 documentation, brahe.space



Access

SUPPORTS

Locations and satellites

Point and polygon ground locations, GeoJSON
Any propagator or trajectory
Many locations and satellites in parallel

Constraints

Elevation, elevation mask, off-nadir, azimuth, range
Local time, look direction, ascending or descending
All, Any, Not, and custom Python constraints

Window properties

Open, close, duration, azimuth, elevation
Doppler, range and range rate, or your own

Every KSAT ground station against one satellite for a day

```python
import brahe as bh
import numpy as np

bh.initialize_eop()
epc = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)

# One satellite at GPS altitude
oe = np.array([bh.R_EARTH + 20_200e3, 0.0, 55.0, 0.0, 0.0, 0.0])
sat = bh.KeplerianPropagator.from_keplerian(
    epc, oe, bh.AngleFormat.DEGREES, 60.0
)

# Every KSAT ground station, with a 10 degree elevation mask
stations = bh.datasets.groundstations.load("ksat")
mask = bh.ElevationConstraint(min_elevation_deg=10.0)

# All access windows over one day
windows = bh.location_accesses(
    stations, sat, epc, epc + 86400.0, mask
)

print(len(stations), "stations,", len(windows), "access windows")
```

```
OUTPUT  36 stations, 65 access windows                    167 ms
```

snippets/mod_access.py · output and run time (imports excluded) from scripts/snippet_outputs.py · brahe 1.8.1 documentation, brahe.space



Data clients

SUPPORTS

GNSS orbit products

IGS products from CDDIS, Garner (SIO) and BKG
SP3 versions a to d read, version d write

Orbit catalogs

CelesTrak GP and SATCAT
Space-Track GP, GP history, SATCAT, CDM
Starlink public ephemerides

Reference data

NAIF SPICE kernels, DE440 and others
Ground station networks: KSAT, AWS, NASA NEN and more
IERS EOP, CSSI space weather, ICGEM gravity

Message formats

CCSDS OEM, OMM, OPM, CDM, APM, AEM

List and read an IGS rapid orbit from the BKG mirror

```python
import brahe as bh

igs = bh.IGSProductsClient("BKG")
epc = bh.Epoch(2026, 11, 1, 12, 0, 0.0, time_system=bh.GPS)

# List the IGS rapid orbit products for the epoch's GPS week
products = igs.list_products(
    epc, center="IGS", solution="RAP", content="ORB", format="SP3"
)

# Download and parse the first one
sp3 = igs.download_sp3(products[0])

print(products[0].name)
print(len(sp3.satellite_ids), "satellites")
```

```
OUTPUT  IGS0OPSRAP_20263050000_01D_15M_ORB.SP3.gz          2.7 ms
        32 satellites
```

snippets/mod_data.py · output and run time (imports excluded, product already in the local cache) from scripts/snippet_outputs.py · brahe 1.8.1 documentation, brahe.space



Estimation

SUPPORTS

Estimators

Extended Kalman filter, Joseph-form update
Unscented Kalman filter
Batch least squares with consider parameters

Measurement models

Inertial and ECEF position, velocity and state
Azimuth, elevation and range from a ground sensor
Custom models in Python

Dynamics and noise

Any force model, with the state transition matrix
Process noise from a user Q matrix, scaled with time

An extended Kalman filter fed thirty position observations

```python
import brahe as bh
import numpy as np

bh.initialize_eop()
epc = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
x0 = np.array([6878e3, 0.0, 0.0, 0.0, 4728.0, 5965.0])  # m, m/s
orbit = bh.KeplerianPropagator.from_eci(epc, x0, 60.0)

ekf = bh.ExtendedKalmanFilter(
    epc,
    x0,
    np.diag([1e6] * 3 + [1e2] * 3),
    measurement_models=[bh.InertialPositionMeasurementModel(5.0)],
    propagation_config=bh.NumericalPropagationConfig.default(),
    force_config=bh.ForceModelConfig.two_body(),
)

# One position observation per minute for thirty minutes
for t in [epc + 60.0 * k for k in range(1, 31)]:
    fix = bh.Observation(t, orbit.state_eci(t)[:3], model_index=0)
    ekf.process_observation(fix)

print(len(ekf.records()), "observations processed")
```

```
OUTPUT  30 observations processed                    21 ms
```

snippets/mod_estimation.py · output and run time (imports excluded) from scripts/snippet_outputs.py · brahe 1.8.1 documentation, brahe.space

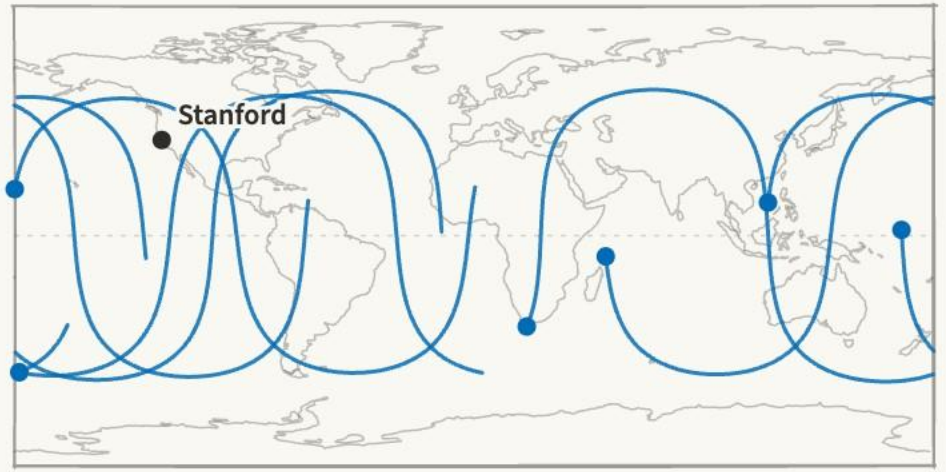
Download public GPS orbit elements and plot ground tracks

```python
import brahe as bh
import numpy as np

bh.initialize_eop()
DEG = bh.AngleFormat.DEGREES
epoch = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
epochs = [epoch + t for t in range(0, 12 * 3600 + 1, 120)]

celestrak = bh.celestrak.CelestrakClient()
records = celestrak.get_gp(group="gps-ops")

tracks = {}
for rec in records:
    sat = rec.to_sgp_propagator(120.0)
    itrf = np.array(sat.states_itrf(epochs))[:, :3]
    geod = [bh.position_ecef_to_geodetic(r, DEG) for r in itrf]
    tracks[rec.object_name] = np.array(geod)[:, :2]  # lon, lat
```



Ground tracks of 6 of the 31 GPS satellites in the public GP catalog, one per orbital plane, starting at the talk time

Public GP element sets, propagated with SGP4 · snippets/ground_tracks.py · full analysis in scripts/quick_hits.py

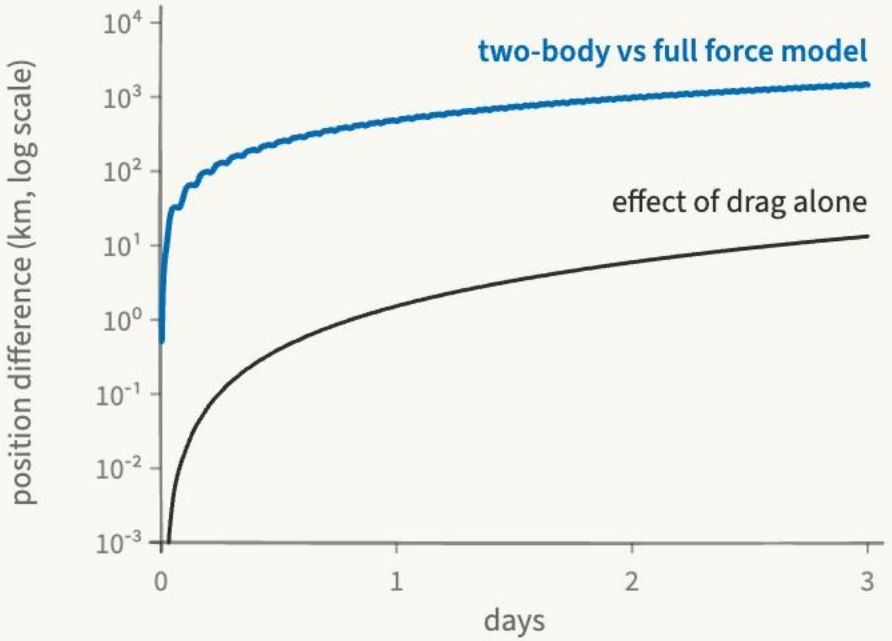
Propagate an orbit with two-body and full force-model dynamics

```python
import brahe as bh
import numpy as np

bh.initialize_eop()
bh.initialize_sw()
DEG = bh.AngleFormat.DEGREES
t0 = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
a = bh.R_EARTH + 500e3
inc = bh.sun_synchronous_inclination(a, 0.0, angle_format=DEG)
x0 = bh.state_koe_to_eci(np.array([a, 0, inc, 0, 0, 0]), DEG)
params = np.array([100, 1, 2.2, 1, 1.3])  # kg, m2, Cd, m2, Cr
cfg = bh.NumericalPropagationConfig.default()

full = bh.ForceModelConfig.default()
no_drag = bh.ForceModelConfig.default()
no_drag.drag = None

r = []
for force in (bh.ForceModelConfig.two_body(), full, no_drag):
    prop = bh.NumericalOrbitPropagator(t0, x0, cfg, force, params)
    prop.propagate_to(t0 + 3 * 86400.0)
    r.append(prop.current_state()[:3] / 1e3)
dr = np.linalg.norm(r[1] - r[0])
dr_drag = np.linalg.norm(r[1] - r[2])
```



500 km sun-synchronous orbit: the two predictions are 1,463 km apart after 3 days, and drag alone moves it 13.3 km

brahe default force model · snippets/force_model.py · full analysis in scripts/quick_hits.py

How far is the public TLE from the precise orbit for the same GPS satellite?

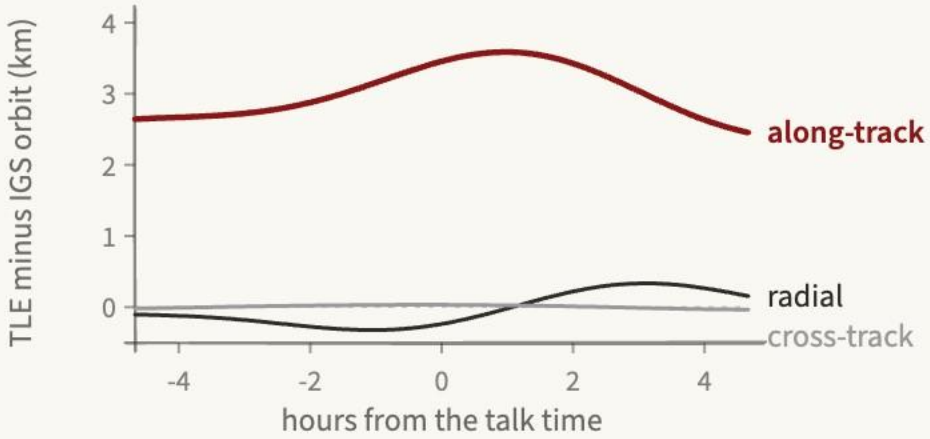
```python
import brahe as bh
import numpy as np

bh.initialize_eop()
epoch = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
epochs = [epoch + t for t in range(-16800, 16801, 300)]
start, end = epochs[0], epochs[-1]

igs = bh.IGSProductsClient("BKG")
issues = igs.list_products_between(
    start, end, center="IGS", solution="ULT", content="ORB"
)
usable = [p for p in issues if p.file_name.start_epoch <= start]
sp3 = igs.download_sp3(usable[-1])  # newest issue
sat = next(s for s in sp3.satellites() if s.get_name() == "G09")

celestrak = bh.celestrak.CelestrakClient()
sgp = celestrak.get_sgp_propagator(catnr=40105, step_size=60.0)

x_sp3 = np.array(sat.states_gcrf(epochs))
x_sgp4 = np.array(sgp.states_gcrf(epochs))
d_rtn = bh.state_eci_to_rtn(x_sp3, x_sgp4)[:, :3] / 1e3
d = np.linalg.norm(d_rtn, axis=1)
rms = np.sqrt(np.mean(d**2))
```



RMS difference over 9.3 hours

3.07 km

Largest difference

3.59 km

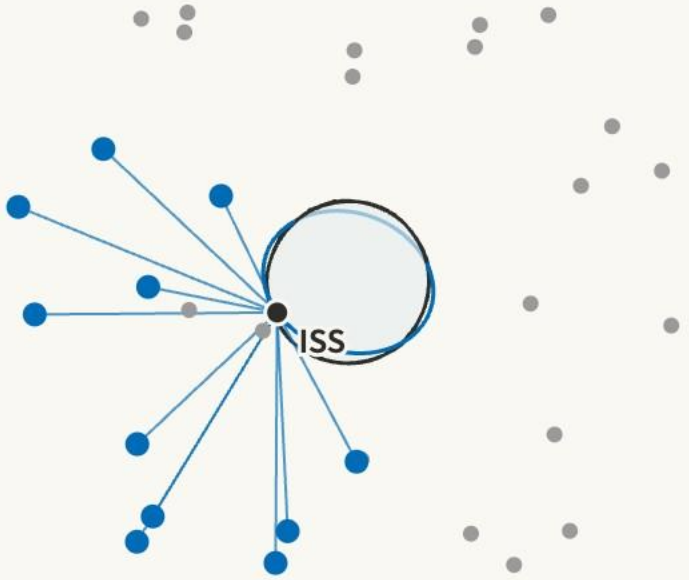
G09 (NORAD 40105): GP elements from Space-Track (epoch November 11, 15:12 UTC) with SGP4, against the IGS ultra-rapid orbit IGS0OPSULT_20263150000_02D_15M_ORB · snippets/tle_vs_sp3.py · full analysis in scripts/tle_vs_sp3.py

Move the receiver from Stanford to the ISS by changing one argument

```python
import brahe as bh
bh.initialize_eop()
DEG = bh.AngleFormat.DEGREES
GEODETIC = bh.EllipsoidalConversionType.GEODETIC
epoch = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
lla = [-122.1697, 37.4275, 30.0]  # Stanford lon, lat, alt
igs = bh.IGSProductsClient("BKG")
ultra_rapid = igs.list_products_between(
    epoch, epoch, center="IGS", solution="ULT", content="ORB"
)
sp3 = igs.download_sp3(ultra_rapid[-1])  # latest issue
iss = bh.celestrak.CelestrakClient().get_sgp_propagator(catnr=25544)

def gps_in_view(rx):
    n = 0
    for sat in sp3.satellites():
        r = sat.state_ecef(epoch)[:3]
        enz = bh.relative_position_ecef_to_enz(rx, r, GEODETIC)
        el = bh.position_enz_to_azel(enz, DEG)[1]
        n += sat.get_name().startswith("G") and el >= 5.0
    return n

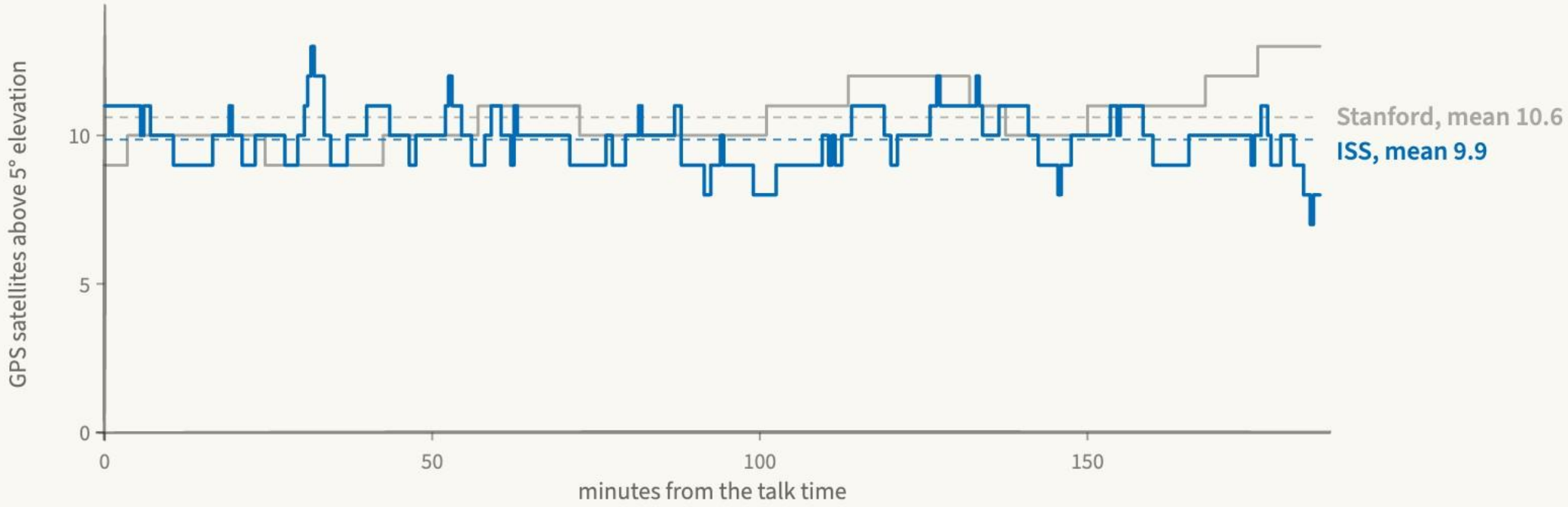
n_ground = gps_in_view(bh.position_geodetic_to_ecef(lla, DEG))
n_iss = gps_in_view(iss.state_itrf(epoch)[:3])
```



The ISS at the talk time, about 425 km up, with lines of sight to the 11 GPS satellites it sees above a 5° mask

ISS GP elements from Space-Track with SGP4 · IGS ultra-rapid orbit IGS0OPSULT_20263150000_02D_15M_ORB · drawn to scale · snippets/leo_receiver.py · full analysis in scripts/leo_receiver.py

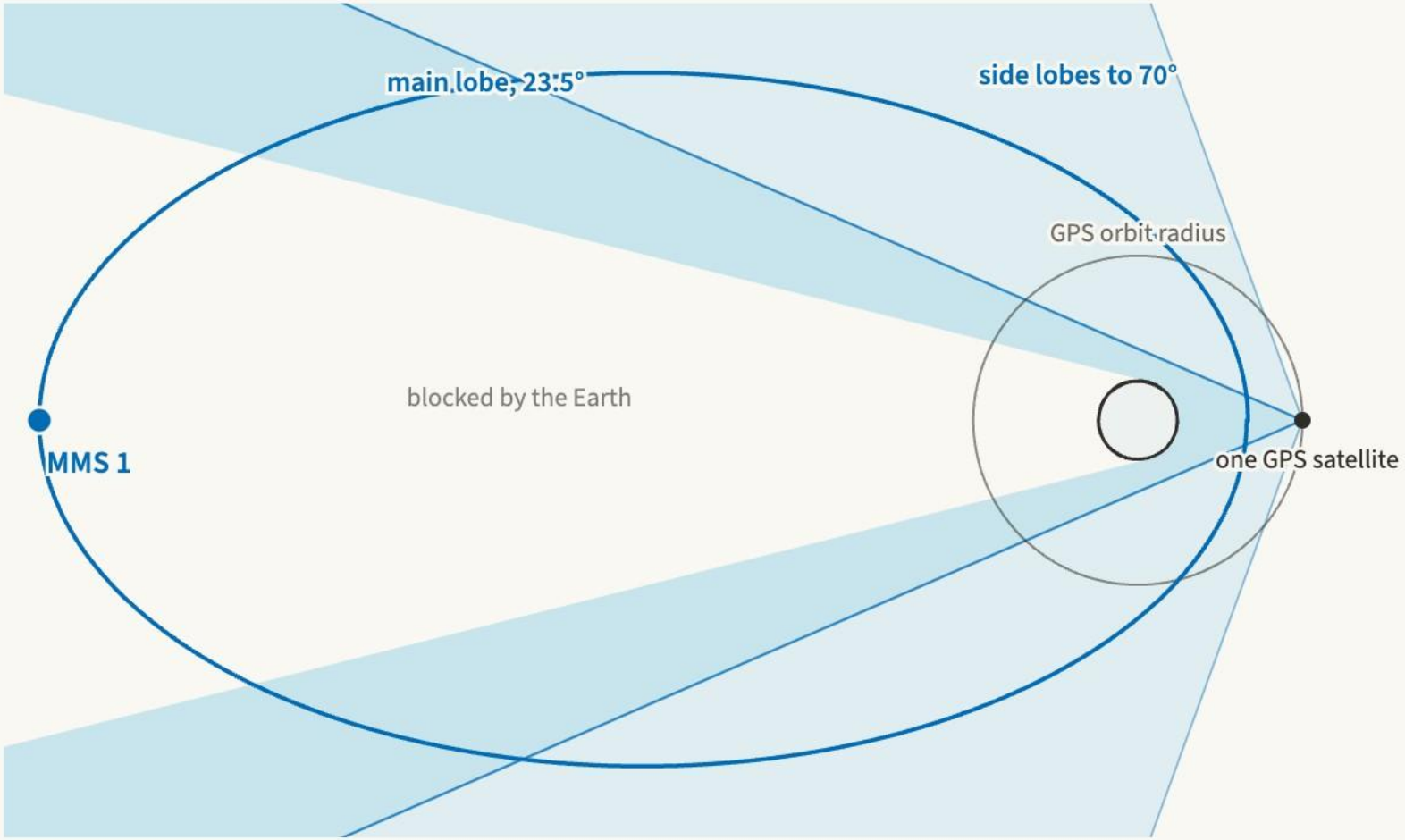
GPS satellites in view of the ground station and the ISS over the next 186 minutes



With the same 5° mask and a zenith antenna, the ISS sees slightly fewer GPS satellites than the ground station: 9.9 against 10.6 on average.

ISS GP elements from Space-Track with SGP4, station on the Stanford campus, one sample every 30 s · scripts/leo_receiver.py

Can you use GPS from above the GPS constellation?



Distance from Earth's center

27.8 Earth radii

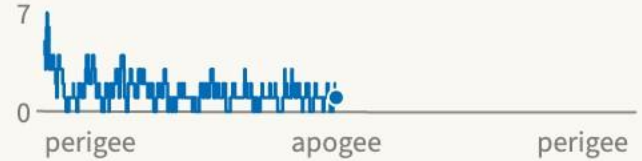
GPS satellites in the main lobe

1

In the main or side lobes

12

GPS satellites in the main lobe over the orbit

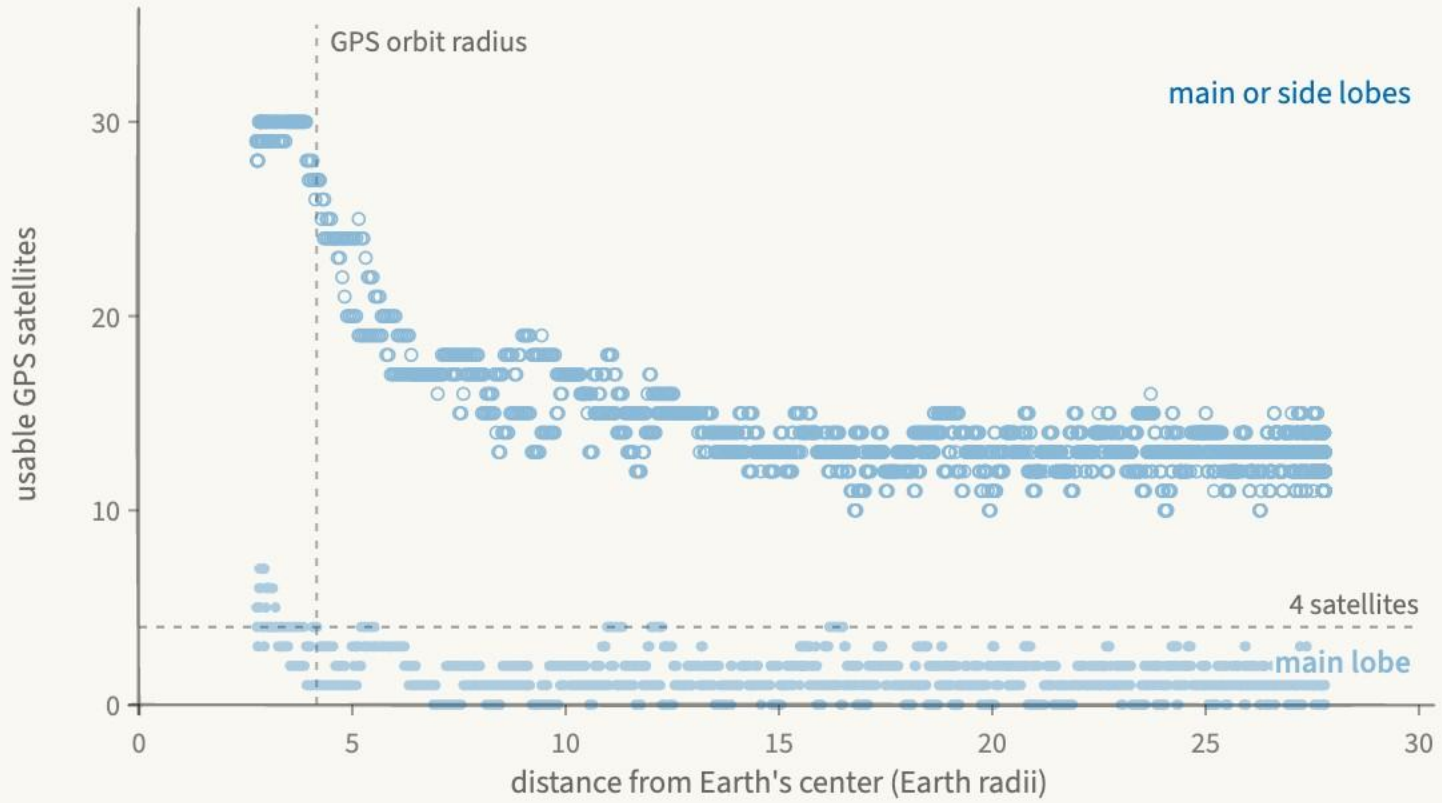


Geometric visibility, no link budget

MMS 1 over one orbit, November 5 to 9, 2026, to scale with the Earth and the GPS orbit radius

MMS 1 GP elements from Space-Track with SGP4; GPS from IGS rapid orbits; transmit pattern after Donaldson et al. (NAVIGATION, 2020; ION GNSS+ 2018) · scripts/mms_visibility.py

GPS satellites usable by MMS 1 against distance from Earth's center



Share of orbit with 4 or more in the main lobe

3.7 %

Share of orbit with 4 or more in main or side lobes

100 %

Geometric visibility, no link budget

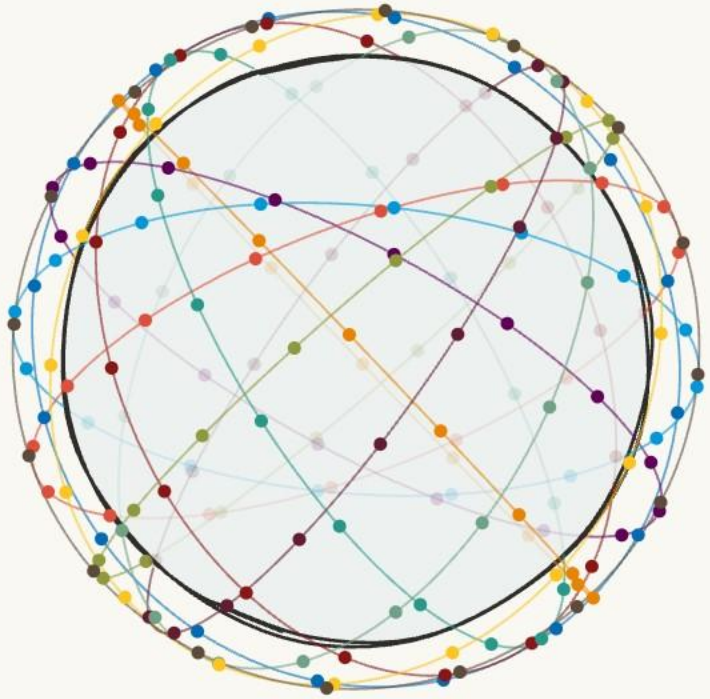
MMS 1 over one orbit, November 5 to 9, 2026, one sample every 2 minutes · scripts/mms_visibility.py

Generate a Walker constellation and propagate every satellite in parallel

```python
import brahe as bh

bh.initialize_eop()
bh.initialize_sw()
t0 = bh.Epoch(2026, 11, 12, 19, 0, 0.0, time_system=bh.UTC)
walker = bh.WalkerConstellationGenerator.builder(
    192, 12, 1, bh.R_EARTH + 1080e3, 53.0, t0
).build()
t_end = t0 + bh.orbital_period(walker.semi_major_axis)
rkf78 = bh.IntegrationMethod.RKF78
cfg = bh.NumericalPropagationConfig.with_method(rkf78)
cfg = cfg.with_abs_tol(1e-6).with_rel_tol(1e-12)
force = bh.ForceModelConfig.default()
params = [100, 1, 2.2, 1, 1.3]  # kg, m2, Cd, m2, Cr

serial = walker.as_numerical_propagators(cfg, force, params)
props = walker.as_numerical_propagators(cfg, force, params)
for prop in serial:
    prop.propagate_to(t_end)
bh.par_propagate_to(props, t_end)
```



Serial loop
2.73 s

Parallel
0.64 s

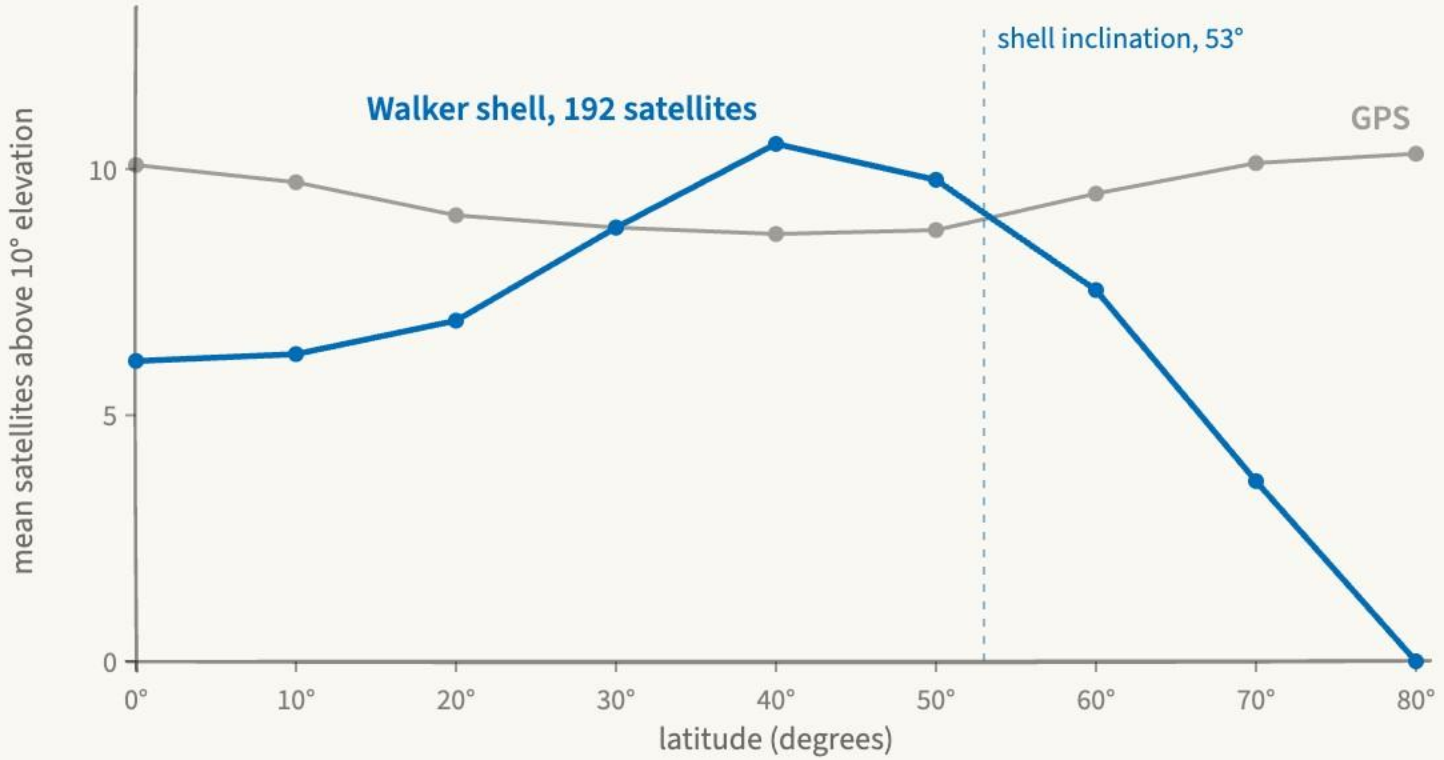
Threads
9

Speedup
4.3 ×

192 satellites in 12 planes at 1,080 km and 53°, propagated for one orbit (107 minutes) with EGM2008 20x20, NRLMSISE-00 drag, SRP (conical eclipse), Sun and Moon third body

Integrator RKF78, abs tol 1e-6, rel tol 1e-12 · wall-clock times on the build machine · snippets/walker.py · full analysis in scripts/walker_pnt.py

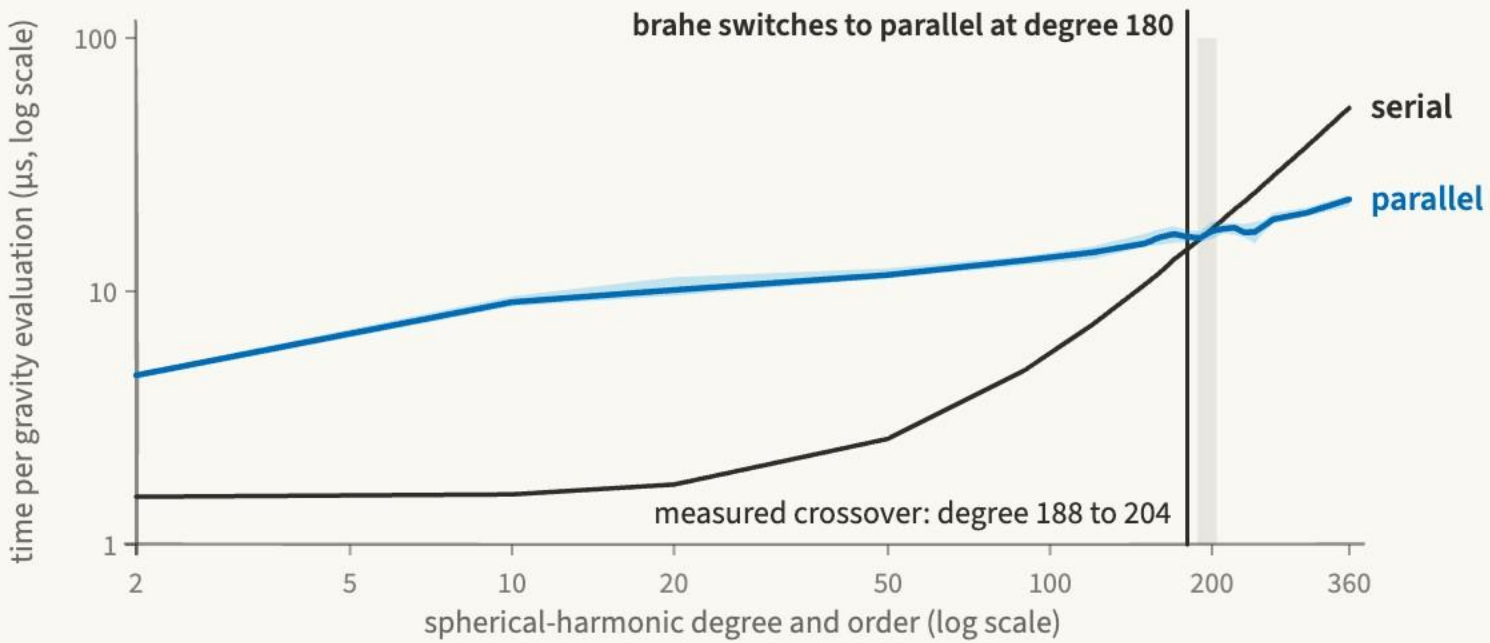
Mean satellites in view by latitude, one Walker shell against GPS



The 53° shell has more satellites in view than GPS between about 30° and 53° latitude and fewer elsewhere: 6.1 against 10.1 at the equator and 0.0 against 10.3 at 80°.

Xona Space Systems, Pulsar constellation (xonaspace.com/technology): 258 satellites at 1080 km, 12 planes of 16 at 53 deg and 6 planes of 11 at 97 deg. From the source: t=192, p=12, 1080 km, 53 deg (the 53 deg shell). Assumed here: Walker delta pattern, phasing f=1, circular orbits, the 97 deg shell (66 satellites) not modeled, and the satellite mass, areas and coefficients.

Time per gravity evaluation against degree and order, serial and parallel



Measured through brahe's numerical propagator with a gravity-only force model (RK4 fixed step 10 s), EGM96 to degree 360, Clenshaw kernel, parallel mode forced off and on. Each value includes about 1.5 µs of integrator and frame-rotation overhead. Lines are medians of 3 passes of 15 runs per degree; bands span the middle half.

Apple M1 Max, brahe thread pool of 9 · scripts/gravity_crossover.py

Degree 10, serial

1.6 µs

Degree 10, parallel

9.1 µs

Degree 360, serial

53.0 µs

Degree 360, parallel

23.1 µs

From degree 180 up, brahe splits the per-order sums across threads automatically.

Median run time of common tasks in other libraries, relative to brahe

UTC to GPS time conversion
per call: 50 epochs

Geodetic to ECEF position
per call: 50 positions

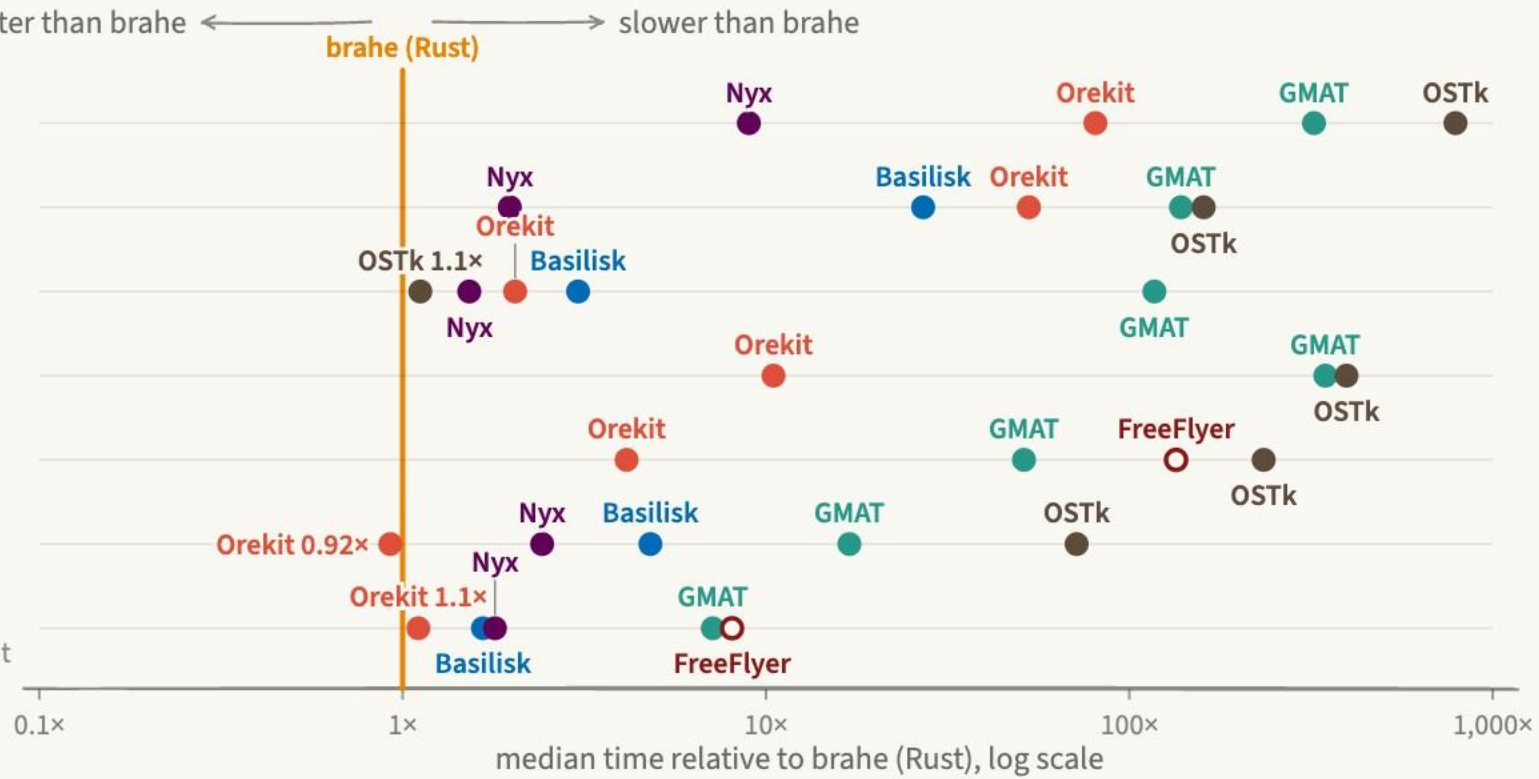
ECI to ECEF state transformation
per call: 50 states

SGP4 trajectory propagation
per call: 1 day at 60 s

Ground-station access windows
per call: 100 sites, 1 day

Numerical propagation, 20x20 gravity, Sun and Moon · per call: about one LEO orbit

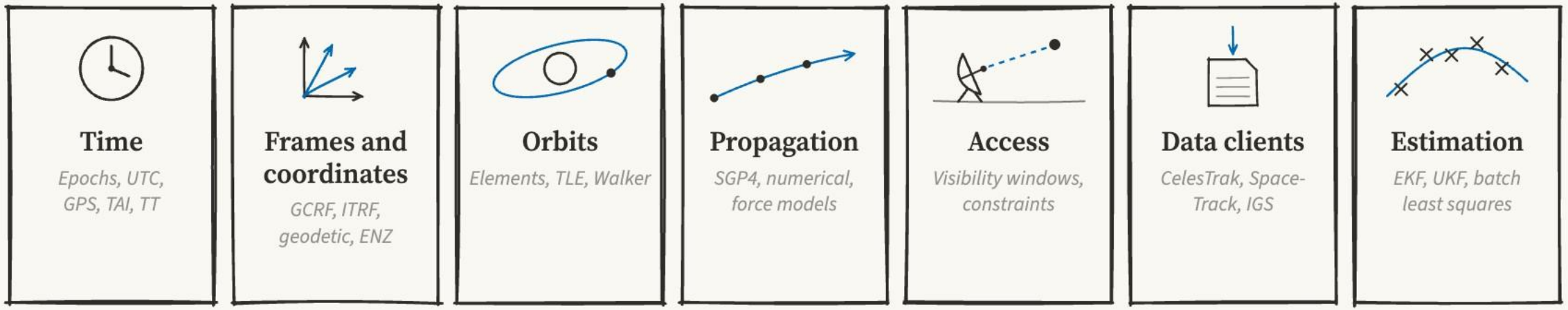
Numerical propagation, 80x80 gravity, Sun, Moon, drag and SRP · per call: about one LEO orbit



brahe from Python takes 1.0 to 2.9 times its Rust time on the five tasks above 100 µs and 20 to 46 times on the two microsecond-scale tasks. Another library is faster than brahe (Rust) on 9 of the 32 tasks in the run, listed in the backup.

brahe comparative benchmark suite, run June 16, 2026 · brahe 1.6.1 development build · AMD EPYC 9554, 64 cores, Ubuntu Linux · median of 200 iterations · Basilisk 2.10.2, GMAT, Nyx 2.4.0, Orekit 13.1.5 and OSTk 17.9.0 · FreeFlyer 7.10.1 (open circle) starts one process per call, about 0.18 s, and is shown where its time is at least 1.5 times that · scripts/benchmarks.py

The modules in brahe, after these examples



Organizations that use brahe



Planned work in brahe, and how you can get involved

On the roadmap

- GNSS orbit determination support, reading RINEX observation and navigation files
- Estimation with a choice of process noise models
- **GPU acceleration:** Astrojax, an experimental GPU-accelerated framework in JAX. SGP4 at 3.3 billion propagations per second on 2 GPUs, about 10 times the Rust sgp4 crate on 128 CPU threads

How you can get involved

1. **Share data:** sources of orbital observations that can be shared for development and testing
2. **Request features** that your work needs
3. **Contribute** code, examples or documentation

SGP4 rates: one satellite, one year at 1 s steps, float64, two NVIDIA H100 NVL GPUs and an AMD EPYC 9554, from github.com/ATTron/astroz/pull/80 · github.com/duncaneddy/astrojax

Every figure in this talk comes from a short script that uses brahe

Duncan Eddy · duncan@cs.stanford.edu

brahe documentation and source

brahe.space

github.com/duncaneddy/brahe

Scripts and slides for this talk

github.com/duncaneddy/2026_spnt_symposium